

ALGORITHM AND DATA STRUCTURE FOR TRIANGULATING MULTIPLY CONNECTED POLYGONAL DOMAINS

LES A. PIEGL and ARNAUD M. RICHARD[†]

Department of Computer Science and Engineering, University of South Florida,
 4202 Fowler Avenue, ENG 118, Tampa, FL 33620

Abstract—A fast algorithm for triangulating multiply connected polygonal domains is presented. The algorithm is an application of the ideas presented in [25], to arbitrary domains. The main contributions are the extension of range searching to handle arbitrary domains and constraint edges, and the generation of the output on the fly without storing triangles that lie outside the valid region.

1. INTRODUCTION

Triangulating an arbitrary (polygonal) domain has many applications. A few examples are (trimmed) surface visualization [1], stereolithography [2], garment design [3], terrain modeling [4], shape reconstruction [5, 6] and finite element analysis [7–9]. Two seminal books [10, 11] deal with the theoretical aspects of triangulation that form the basis of the many papers [12–24] that have been published on domain triangulations (if the domain is singly connected and is convex, the domain triangulation problem reduces to the simple triangulation problem to which the best known solution is the Delaunay triangulation). This paper extends the ideas, presented in a previous paper [25] for Delaunay triangulation, to triangulating multiply connected domains. The domain is defined by a set of polygons oriented as shown in Fig. 1. More specifically, we look at the following problem:

Given a multiply connected polygonal domain, a set of points (call them *nodes*) lying inside the domain, and a set of line segments (call them *constraint edges*) connecting pairs of nodes. Obtain a triangulation satisfying the following conditions:

- the vertices of the triangles are the nodes as well as the vertices of the polygons,
- no triangle contains a node other than its vertices,
- the interiors of the triangles are pairwise disjoint,
- the union of the triangles is the multiply connected domain,
- if the circumcircle of any triangle contains a node other than the vertices of the triangle, then every interior point of the triangle is separated from this point by either a constraint or a boundary edge (call this the *Modified CircumCircle (MCC)* criterion [24]),
- each constraint edge belongs to two triangles, and each side of the boundary polygon belongs to one and only one triangle.

Condition (b) ensures that the algorithm triangulates the domain properly whereas conditions (c) and (d)

are necessary to ensure that the constraints are satisfied as well.

We present a constructive, algorithmic solution to this problem. We also discuss implementation details. It is our experience that implementation is not at all a trivial exercise, and without it the study of an algorithm is incomplete. Our algorithm consists of three major parts:

- Range searching*: given an edge, find a point to form a valid triangle.
- Shelling*: put triangles together so that completeness and correctness are maintained.
- Output generation*: create an output data structure without storing excess triangles (triangles that do not belong to the valid region).

The organization of the paper is as follows. In Section 2 it is shown how to preprocess the data and how to build an intermediate data structure. In Section 3 range searching is explained, and in Section 4 details of the shelling procedure are given. Section 5 explains how the output data structure is computed. Finally, in Sections 6 and 7 examples and conclusions, respectively, are presented. Unless otherwise specified, both boundary and constraint edges are called constraint edges in the rest of the paper. This does not cause confusion as both edges impose constraints on the triangulation.

2. DATA STRUCTURES

Assume that we are given a set of points

$$(x_i, y_i), i = 0, \dots, n,$$

and a set of directed lines

$$L_i = [(x_k, y_k), (x_\ell, y_\ell)],$$

$$i = 0, \dots, N, \quad 0 \leq k, \quad \ell \leq n.$$

We assume that the lines L_i either form a multiply connected domain or connect pairs of nodes, as illustrated in Fig. 1. The following operations are crucial to perform fast when the algorithm is running:

- Given an edge, find a point in the neighborhood of the edge to form a valid triangle.

[†] On leave from Renault Direction de la Recherche, France.

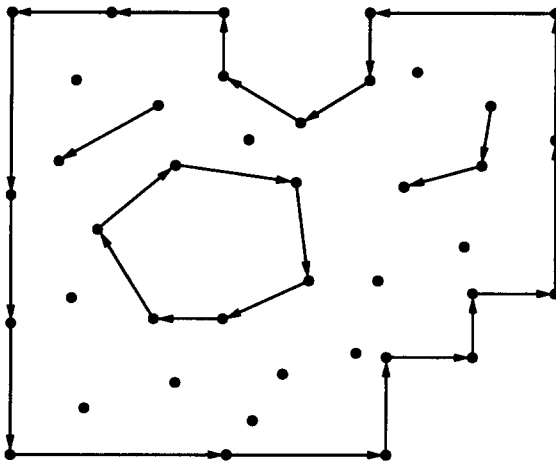


Fig. 1. Multiply connected polygonal domain with constraint edges.

- Given an edge, find all constraint edges that intersect this edge.

To make these *database browsing*, i.e., neighborhood searching, operations fast, we elected to put the above data into a uniform grid data structure. The idea is explained as follows:

1. Compute the min-max box of the data set.
2. Compute the grid size such that in the average no more than one point lies in one cell.
3. Place each point in one and only one cell (the min-max box needs to be offset by the point coincidence tolerance to avoid conflict if the point lies along grid lines).
4. For each constraint edge, find all the cells that intersect that line.

To avoid problems with coincident points, before each point is put into a cell, the cell is checked whether the candidate point coincides with any of the points currently in the cell. If yes, then the candidate point is ignored. Otherwise it is put into the cell.

After the above preprocessing is done, we build two data structures: a *point list* and a *constraint edge list*, Fig. 2. The point list is actually a two-dimensional array of pointers pointing to linked lists. That is, each cell is associated with a linked list whose nodes contain points that lie in the cell. The structure definition of each node is as follows:

```
typedef struct ptnode
{
    int vertex_number;
    double x, y;
    int used;
    int boundary;
    struct ptnode *next;
    struct ptnode *next_node;
} PTNODE;
```

where

- **vertex-number** is the ID number of the point (x, y),

- **used** is a flag to indicate whether the point was used to form a triangle (initially zero and if the point is found, it is set to 1),
- **boundary** is a flag to indicate whether the point lies along the boundary or not (it is set zero for non-boundary points and 1 for boundary points), and
- **next** is a pointer used for boundary points or for points connected by constraint edges to point to the end point of the directed edge (it is NULL for nodes not lying either on the boundary or on constraint edges).

The constraint edge list is also a two-dimensional array of pointers pointing to linked lists. These lists, however, are simply lists of pointers pointing to edge points in the point lists (Fig. 2). The nodes have the following structure definition:

```
typedef struct edgenode
{
    PTNODE *edge;
    struct edgenode *next_node;
}
```

That is, each constraint edge is defined by its start point only. The advantage of this representation is twofold. First, the end points of the constraint edges are stored only once. Second, in case the constraint edges have to be split, because some data points lie on them, the split entails simple pointer assignments only and no data movement is necessary.

3. RANGE SEARCHING

Range searching considers each edge, stored in a temporary edge list, called the shell edge list, and searches a certain area of the 2-D domain to find a point that forms a triangle with the given edge. To start range searching, the first edge needs to be found. This is done by picking the first point, that is more or less in the middle of the data set, and computing the closest point to it. The closest point algorithm is as follows (Fig. 3):

```
dmin ← diagonal of the min-max box;
Ps ← start point;
if ( Ps.next != NULL ) { Pn ← Ps.next; return; }
while ( not done )
{
    W ← found point;
    d ← dist ( Ps, W );
    if ( line ( Ps, W ) !intersect ( edges ) )
    {
        if ( dmin < d ) { dmin ← d; Pn ← W; }
    }
}
```

If the start point happens to be a point of any of the constraint edges, then the constraint edge is chosen (edge $\langle P_s^2, P_n^2 \rangle$ in Fig. 3). This is correct as these edges must be included into the triangulation. However, if the point is a node not lying on any of the

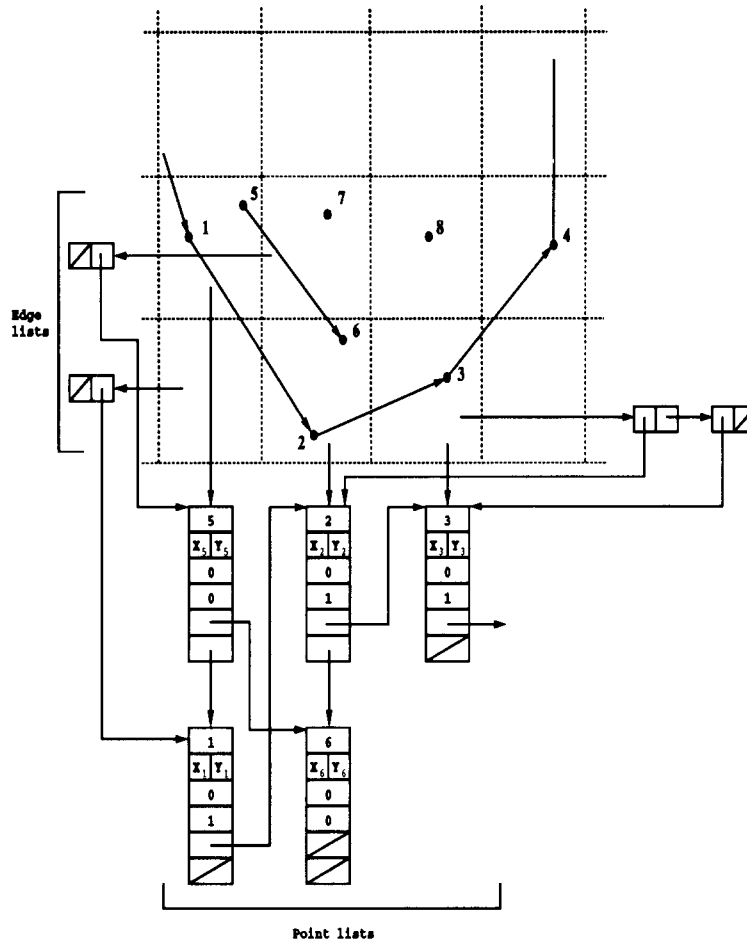


Fig. 2. Data structures to store points and constraint edges.

constraint edges, then the nearest point is searched for such that the line $\langle P_s, P_n \rangle$ does not intersect any constraint edge (the nearest point to P_s is P_n^1 in Fig. 3). The intersection check is done very efficiently. Assume that the candidate line $\langle P_s, W \rangle$ intersects or lies inside a cell. The line is checked against those edges that are in the linked list of this cell. This cuts down significantly on the number of candidate edges the algorithm needs to check. Since the cell size is adjusted to the number of points, *i.e.*, the more points we have, the smaller the grid size, and since the number of constraint edges is roughly proportional to the number of points, we may say that the number of constraint edges in the linked list of a cell is a constant. That is, if the nodes and the constraint edges are not jammed into one spot of the plane, *i.e.*, they are more or less uniformly distributed, then the intersection check is done in practical $\Theta(1)$ time.

As a practical matter, the algorithm searches the start point's cell first to see if points are inside. If yes, then the nearest point is computed (it always checks if the current edge $\langle P_s, W \rangle$ intersects any of the edges in the constraint edge list). Next, the search continues in the neighboring cells as depicted in Fig. 3 (first in the top 3 cells, in the bottom 3 cells, and in the left and right cells—then in the top 5 cells, in the bottom 5

cells, and in the left and right 3 cells, *etc.*). The search stops whenever the minimum distance is smaller than any of the perpendiculars dropped from P_s to the sides of the currently searched rows and columns nearest to P_s (this is the point when the **while** loop is done).

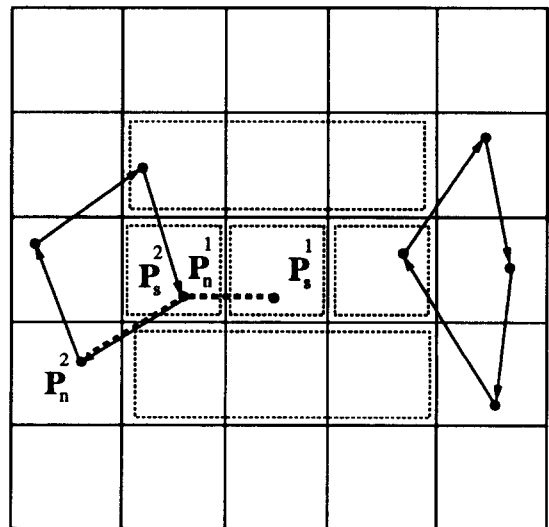


Fig. 3. Finding the first edge.

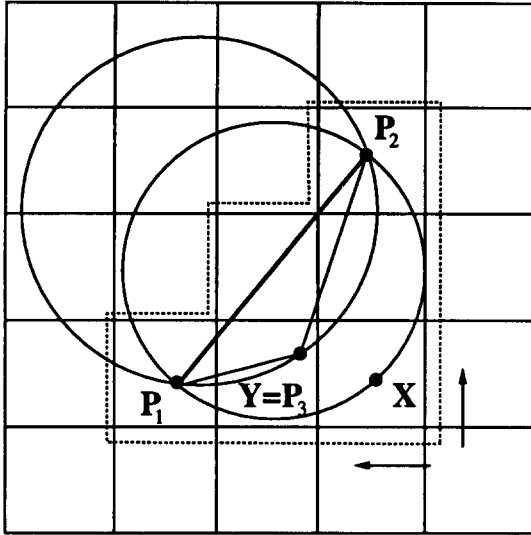


Fig. 4. Range searching to find a triangle given an edge.

Once the first edge is found, range searching is applied to create the first triangle. Assume that the edge $\langle P_1, P_2 \rangle$ is found. A point P_3 is sought such that the triangle $\langle P_1, P_2, P_3 \rangle$ does not intersect any constraint edge and satisfies the MCC criterion. The algorithm is as follows (Fig. 4):

1. Search on the right hand side of $\langle P_1, P_2 \rangle$.
2. Find the cells that are either the cells of the end points or their immediate neighbors. These cells can contain points that are on the right hand side of the line P_1, P_2 .
3. Form a triangular area covered by cells. The vertices along the line $\langle P_1, P_2 \rangle$ are the cells computed in the above step, and the boundaries are cells arranged horizontally, vertically and diagonally (the dotted area in Fig. 4).
4. Check each cell in the triangular area. If points are found, then
 - (a) choose the one (point Y in Fig. 4) that satisfies two conditions:
 - i. the angle $\angle(P_1, Y, P_2)$ is the largest, *i.e.*, it results in the smallest cosine, and
 - ii. the lines $\langle P_1, Y \rangle$ and $\langle Y, P_2 \rangle$ do not intersect any constraint edge.
 - (b) get the circle defined by P_1, P_2 and the point with the minimum cosine, *i.e.*, point Y , and
 - (c) set the min-max box of the circle.[†] Otherwise, grow the triangular area outwards and search the new row and column of cells.
5. After the first min-max box is set, continue searching along rows and columns inside the min-max box and do the same thing as above, *i.e.*, select the point that satisfies the minimum cosine and the no in-

tersection criteria, draw the circle and update the min-max box.

6. The search stops when there are no unvisited rows and columns that are inside the current min-max box.

The dynamic update of the circle bounding box is an essential part of the algorithm as points close to the line $\langle P_1, P_2 \rangle$ can result in huge boxes that might lead to searching for points in the entire data set. The main goal is to limit the search to the immediate vicinity of $\langle P_1, P_2 \rangle$ so that points are found within a few steps.

If the edge $\langle P_1, P_2 \rangle$ happens to be horizontal or vertical, then the above algorithm is simplified by using simple boxes instead of triangles. For more details, see [25].

After the first triangle is computed, range searching is repeated with the newly created edges to build new triangles around the existing one. In the Appendix, a simple proof is given that range searching, as described above, computes triangles that satisfy the MCC criterion.

4. SHELLING

The most crucial part of the triangulation is the one that puts triangles together so that completeness and correctness are maintained, that is, the entire domain is triangulated and no holes are created. Our method that performs this is called *shelling*. First the definition of the shell (Fig. 5):

The shell is a triangulated region that satisfies the following requirements:

- R1:** the triangles inside the shell are a subset of the final triangulation,
- R2:** each point along the boundary of the shell connects to exactly two other shell points, and
- R3:** the boundary of the shell is simply connected and forms a closed region.

Obviously, multiply connected domains are unable to satisfy requirement **R3** above. To make the shelling applicable to domains with holes, the algorithm “pretends” that the domain is convex and is singly con-

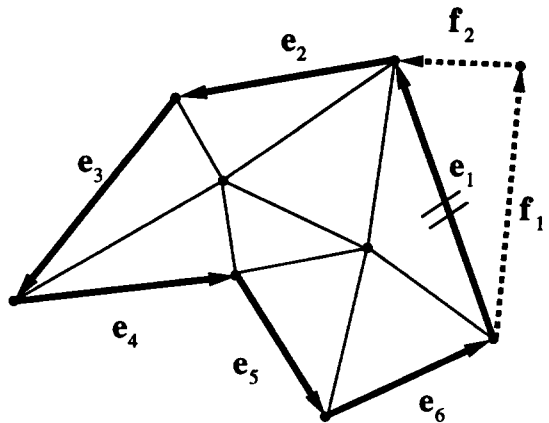


Fig. 5. Shelling.

[†] The sides of the box are bounded by grid lines.

nected. That is, triangles *are* computed outside the valid domain *and* are put into a *temporary* edge list, the shell edge list, to ensure correctness of the triangulation. However, output is not generated for triangles outside the domain. That is, assume that range searching returned two new edges. They are put into the shell edge list, and if they are completely inside the domain, they are appended to the output list. Otherwise, no output is created with these edges.

The procedure of shelling works like this (Fig. 5):

1. Given a shell boundary defined by the edges $\langle e_1, e_2, \dots, e_i, \dots, e_n \rangle$, oriented in counterclockwise order (these edges form the shell edge list).
2. For each edge e_i , form a triangle, *i.e.*, create two new edges f_1 and f_2 .
3. Erase e_i from the shell edge list and add f_1 and f_2 to the list so that the new boundary is formed by the edges $\langle e_1, e_2, \dots, e_{i-1}, f_1, f_2, e_{i+1}, \dots, e_n \rangle$.
4. When searching on the right-hand side of each edge, if no points are found, then the edge is part of the convex hull and is removed from the shell edge list.
5. The shelling procedure stops whenever the shell edge list becomes empty, at which point the triangulation is complete.

There are two points that need to be discussed: (1) what is the start shell, and (2) how to ensure correctness. The start shell is simply the first edge which we call a *null shell*. More specifically, if P_1 is the start point and P_2 is the nearest point forming the first edge $\langle P_1, P_2 \rangle$, then the null shell consists of two edges $\langle P_1, P_2 \rangle$ and $\langle P_2, P_1 \rangle$. Once the first point P_3 is found, the current edge $\langle P_1, P_2 \rangle$ is removed and the new edges $\langle P_1, P_3 \rangle$ and $\langle P_3, P_2 \rangle$ are added to the list (in that order). Using $\langle P_1, P_3 \rangle$ as the current edge, the procedure is repeated as explained above.

To ensure correctness, the following problems need to be solved:

1. One of the newly created edges overlaps with one of the shell boundary edges [Fig. 6 (we call this a

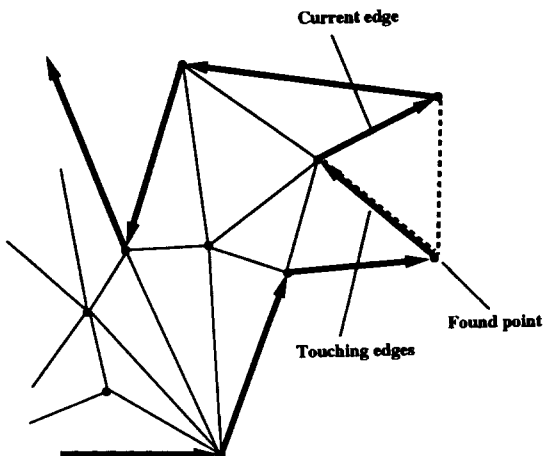


Fig. 6. Computed triangle touches shell boundary.

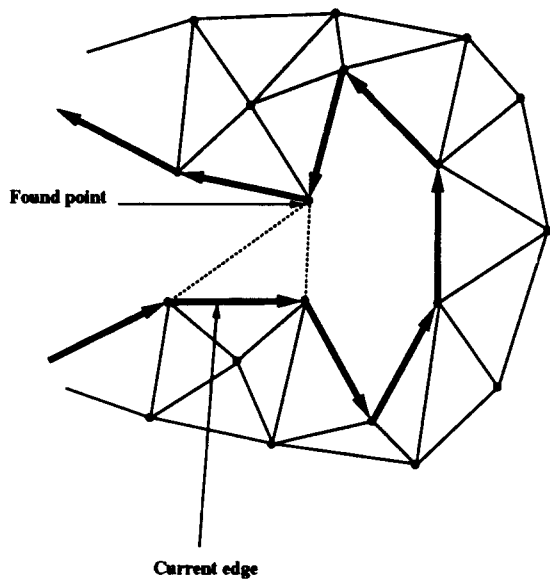
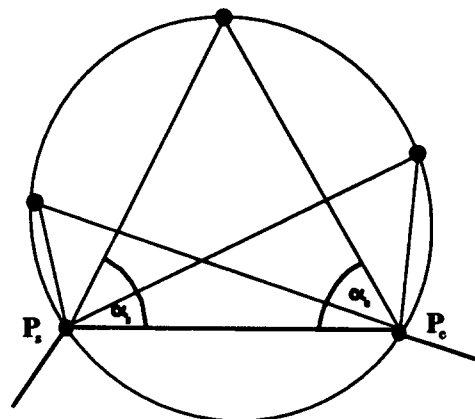


Fig. 7. The no touch case.

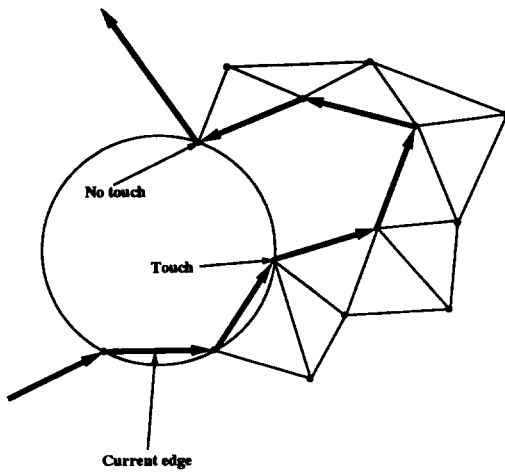
touch case)). A simple fix is to remove both edges from the shell edge list.

2. The point used to form a new triangle is a shell boundary point (*i.e.*, it is a *used* point), however, none of the two new edges touches the shell boundary (Fig. 7). This is a very crucial case as it results in a hole if not detected. In this case the current edge is simply skipped (but not removed) and the next in the edge list is considered.
3. Several points are found to yield the minimum cosine. There are two cases to consider:
 - (a) None of the points have their *used* flag set (Fig. 8). We elected to choose the one that yields the minimum base angle difference.
 - (b) Some or all of the points are *used* (Fig. 9). If we have a *no touch* case only, then the edge is



$$|\alpha_1 - \alpha_2| = \text{minimum}$$

Fig. 8. Co-circular points with no *used* nodes.

Fig. 9. Co-circular points with *used* nodes.

skipped. However, if there is a touch case, then the point where the touch happens is chosen (if there are more than one touch points, then the one next to the end point of the current edge is picked. Note that we can have touch cases only at one end of the current edge, because if we had touch cases at both ends, that they would create a hole in the triangulation).

5. OUTPUT

In order for the triangulation to be useful for further processing, *e.g.*, trimmed surface representation by faceting or contouring of terrain surfaces, the computed triangles must be stored in an output data structure. The data structure must not only store triangles, however, it should incorporate neighboring information. The structure that our algorithm uses is based on a point list introduced in the next chapter.

5.1. Output data structure

Consider the example shown in Fig. 10. Two point lists are used; in the first all interior points, whereas in the second all boundary points are listed. For each point all surrounding points are listed in clockwise order. The data structure representing the triangles, the edges and the points of the triangulation shown in Fig. 10 is as follows:

1	7	6	2	8		
2	6	5	4	3	8	1
3	8	2	4			
4	3	2	5			
5	4	2	6			
6	5	2	1	7		
7	6	1	8			
8	7	1	2	3		

The points in the first column below the double line are boundary points, whereas those above the double line are internal ones. The above data structure allows answering all geometric queries. A few examples are:

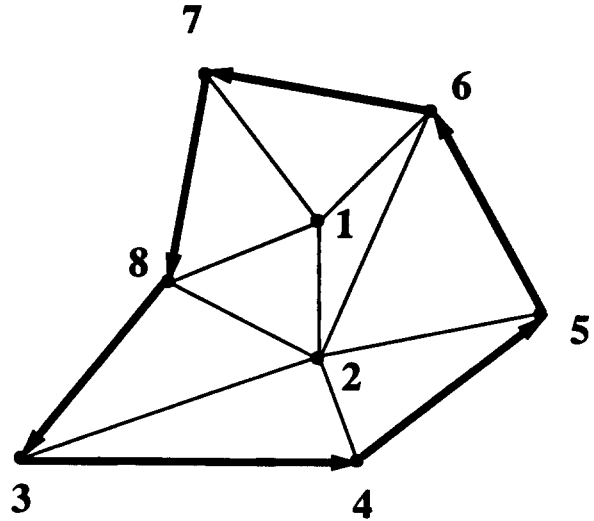


Fig. 10. Sample triangulation to illustrate output data structure.

- *All triangles incident at a point.* For point 1: $\langle 1, 7, 6 \rangle$, $\langle 1, 6, 2 \rangle$, $\langle 1, 2, 8 \rangle$ and $\langle 1, 8, 7 \rangle$ (note that in case of internal points, the list is circular). For the boundary point 3: $\langle 3, 8, 2 \rangle$ and $\langle 3, 2, 4 \rangle$.
- *All triangles incident on an edge.* For edge $\langle 3, 2 \rangle$ one gets $\langle 3, 2, 4 \rangle$ from 3's list and $\langle 2, 3, 8 \rangle$ from 2's list. If the edge is $\langle 3, 4 \rangle$, then from 3's list it follows that $\langle 3, 4 \rangle$ is a boundary edge, and from 4's list $\langle 4, 3, 2 \rangle$ is obtained.
- *All edges incident at a point.* For 4: $\langle 4, 3 \rangle$, $\langle 4, 2 \rangle$ and $\langle 4, 5 \rangle$.
- *All triangles neighboring to a given triangle.* If the given triangle is $\langle 1, 2, 8 \rangle$, then the triangles incident on the edges $\langle 1, 2 \rangle$, $\langle 2, 8 \rangle$ and $\langle 8, 1 \rangle$ are sought. From 1's list we get $\langle 1, 6, 2 \rangle$, from 2's list $\langle 2, 3, 8 \rangle$ is obtained and from 8's list $\langle 8, 7, 1 \rangle$ is computed.

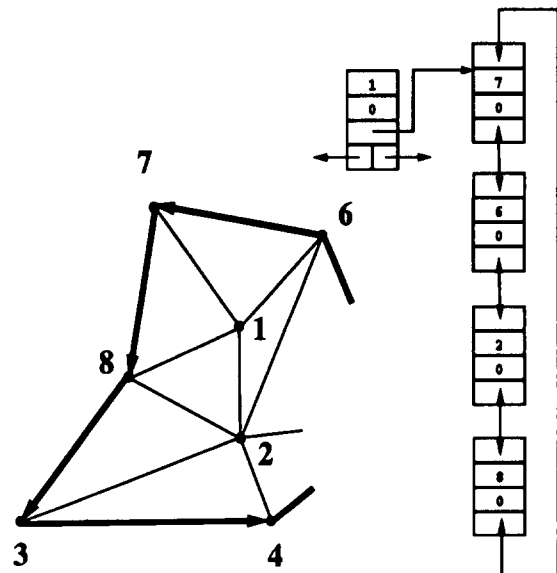


Fig. 11. Point list to store neighbors of an internal point.

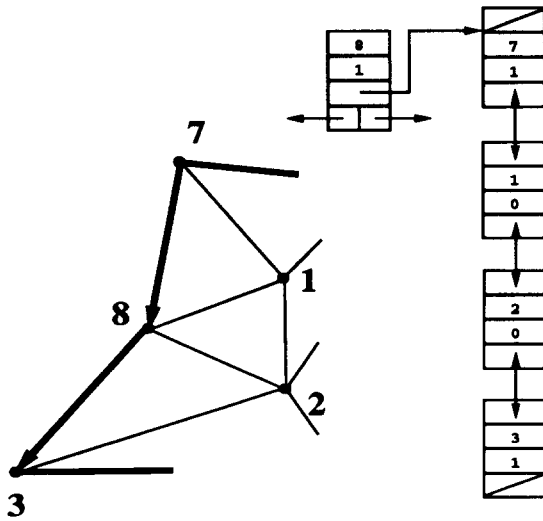


Fig. 12. Point list to store neighbors of a boundary point.

The above data structure is very compact. Simple sub-routines can be written that perform data structure browsing operations.

The data structures to represent internal and boundary point lists are illustrated in Figures 11 and 12, respectively. The structure definitions are as follows:

```
typedef struct header
{
    int point_number;
    int boundary;
    PTLIST *start;
    struct header *prev, *next;
} HEADER;
```

```
typedef struct ptlist
{
    int point_number;
    int boundary;
    struct ptlist *prev, *next;
} PTLIST;
```

Both the headers as well as the individual point lists are placed in double linked lists for ease of insertion and deletion. During the triangulation both the boundary and the internal point lists are circular and the points are listed in clockwise order. After the triangulation is complete, the boundary point list becomes noncircular. The flags used to distinguish between boundary and internal edges are assigned as follows:

- the header point receives a flag zero if it is internal, and receives one if it is external.
- all the points in the list receive flags based on whether the line connecting them to the header is boundary or not.

For example, the header point 1 in Fig. 11 and all the incident edges receive the flag 0. The situation depicted

on Fig. 12 is different using the boundary point 8. Since the edge $\langle 8, 7 \rangle$ is boundary edge, the point 7 gets a flag 1, whereas the point 1 receives a flag 0 since the edge $\langle 8, 1 \rangle$ is an internal edge.

5.2. Output generation

The output is computed on the fly, i.e., as soon as a new triangle is computed, both the shell edge list (used to control the shelling procedure) and the output lists are updated at the same time (note that for regular Delaunay triangulation only one list is needed since all triangles that are computed are part of the output [25]). First, let us look at how the output lists are updated as triangles are added and checked for touch cases. Assume that the start edge is $\langle 1, 2 \rangle$. The initialization of the output lists is:

F	1	2
L	2	1

where F and L are the first and last lists in the output structure. The current edge is $\langle 1, 2 \rangle$ which is $\langle F(1), L(1) \rangle$ [$F(1)$ and $L(1)$ denote the first elements in the lists of F and L]. After point 3 is found, the list is updated as follows:

1. append 3 to the list of F ,
2. insert 3 to the second position of L 's list,
3. create a new list with 3 as the header and append $F(1)$ and $L(1)$ to it, and
4. advance the pointer L .

The new list becomes:

F	1	2	3
	2	3	1
L	3	1	2

The current edge is $\langle F(1)L(1) \rangle = \langle 1, 3 \rangle$ and the new point 4 is found. Following the same procedure, the edge list becomes:

F	1	2	3	4
	2	3	1	
	3	4	1	2
L	4	1	3	

The above *insert*, *append* and *create* operations are repeated as new triangles are added to the output. Modifications are necessary in the following cases:

- a *touch* case is detected,
- a *used* point is found, however, no touch case is detected, or
- no point is found, i.e., the current edge was part of the convex hull.

Based on the shell edge list discussed above, there are two touch cases: (1) one of the newly created edges touches the one that is at the front of the edge list (this is called a *right touch*), and (2) one of the edges touches

the one that is at the second position from the rear edge in the edge list (this is called a *left touch*). In case of the right touch the output list is updated as follows:

1. append $L(1)$ to the end of the list following F ,
2. insert $F(2)$ to the second position of L , and
3. advance the pointer F .

For left touch, the procedure is:

1. append $L(\text{last})$ to the end of F 's list,
2. decrement the pointer L , and
3. insert $F(1)$ to the second position of the (new) list of L .

In case of the no touch, the current edge is skipped and the one at the front is moved to the rear to become the current edge in the shell edge list. This is simply done by incrementing both F and L pointers (remember that the lists are circular). The same procedure is applied in case of an edge that is part of the convex hull.

Below we modify the above method to account for boundary edges, in particular, to account for concave polygons. To make the algorithm easy to understand, we walk the reader through the critical case shown in Fig. 13.

Assume that the first edge was $\langle 3, 4 \rangle$ and that the triangles $\langle 3, 4, 5 \rangle$ and $\langle 3, 5, 6 \rangle$ have been created. The point lists are:

F	3	4	5	6
	4	5	3	
	5	6	3	
L	6	3	5	

The current edge is $\langle 3, 6 \rangle$, the point 1 is found, and two new edges $\langle 1, 6 \rangle$ and $\langle 3, 1 \rangle$ are created. Since both edges are boundary edges, we do the following:

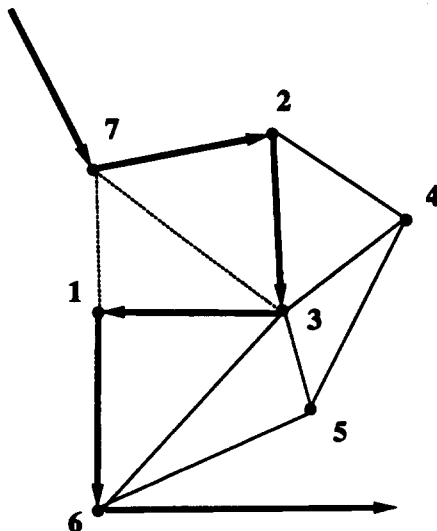


Fig. 13. Computing and storing triangles near the boundary.

1. Append point 1 to the end of 3's list.
2. Since we approached the boundary from inside, any point that is found on the right hand side of $\langle 3, 1 \rangle$ should not be appended to 3's list. Consequently, 3 gets a "no append" (**na**) flag till we enter the inside of the boundary again.
3. Point 1 is inserted into $L(2)$. Since $\langle 1, 6 \rangle$ is a boundary edge, the point 6 gets a "no insert" (**ni**) flag.
4. A list for 1 is created with points 3 and 6, and it gets both the **na** and the **ni** flags as both $\langle 1, 3 \rangle$ and $\langle 1, 6 \rangle$ are boundary edges.

The point lists become:

F	3 ^{na}	4	5	6	1
	4	5	3		
	5	6	3		
	6 ⁿⁱ	1	3	5	
L	1 ^{na, ni}	3	6		

The current edge is $\langle 3, 1 \rangle$ and the point 7 is found. Since point 3 has the **na** flag, point 7 is not appended to its list. Similarly, since 1 has the **ni** flag, 7 is not inserted into $L(2)$. A new list is created with 7, however, as no valid triangle is created, no points are listed and 7 gets both the **na** and the **ni** flags. The new list is:

F	3 ^{na}	4	5	6	1
	4	5	3		
	5	6	3		
	6 ⁿⁱ	1	3	5	
	1 ^{na, ni}	3	6		
L	7 ^{na, ni}				

The current edge then becomes $\langle 3, 7 \rangle$, point 2 is found and two edges $\langle 2, 7 \rangle$ and $\langle 3, 2 \rangle$ are created. The found boundary point, that is 2, is processed as follows:

1. Append 2 to 3's list. Since we entered the domain again, the **na** flag is canceled.
2. Insert 2 into $L(2)$ and cancel the **ni** flag.
3. Create 2's list by adding 3 and 7. Since $\langle 2, 3 \rangle$ and $\langle 2, 7 \rangle$ are both boundary edges, no **na** flag is set.

This yields the new point list:

F	3	4	5	6	1	2
	4	5	3			
	5	6	3			
	6 ⁿⁱ	1	3	5		
	1 ^{na, ni}	3	6			
	7 ^{na}	2				
L	2	3	7			

The current edge is $\langle 3, 2 \rangle$. Point 4 is found and a right touch is detected. As detailed above, it is processed as follows:

1. append $L(1)$ to the end of the list following F ,
2. insert $F(2)$ to the second position of L , and
3. advance the pointer F .

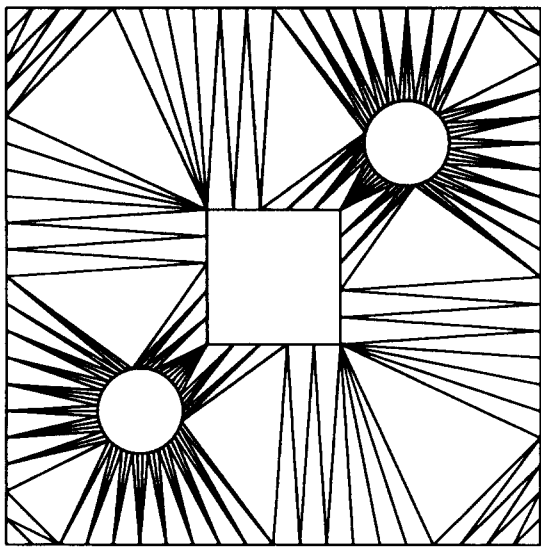


Fig. 14. Triangulation example: 200 edges and no internal points.

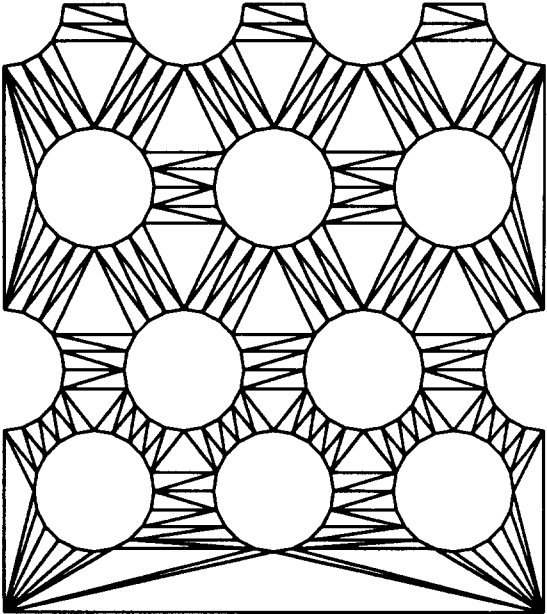


Fig. 16. Triangulation example: 219 edges and no internal points.

The point list after right touch processing looks like this:

	3	4	5	6	1	2
F	4	5	3	2		
	5	6	3			
	6^{ni}	1	3	5		
	$1^{na,ni}$	3	6			
	7^{na}	2				
L	2	4	3	7		

The current edge is $\langle 4, 2 \rangle$ and the procedure continues until the shell edge list becomes empty at which point the triangulation is complete. Note that the point list of 3 is not as efficient as those of Fig. 10. Since at this

point the list is circular, it is easy to make the header point to 2 and the 1's pointer to point to NULL. This way the list becomes noncircular and only triangles that are inside the domain will be traversed. The circular version of the list was necessary to make the output generation easy to handle.

6. EXAMPLES

We have tested our algorithm with numerous data sets containing many internal points and constraint

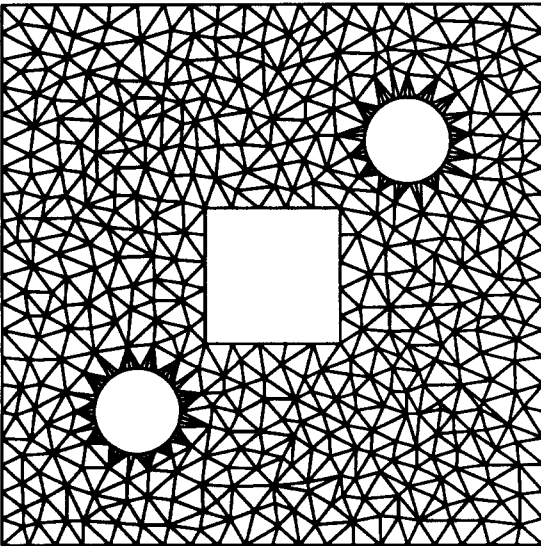


Fig. 15. Triangulation example: 200 edges and 400 internal points.

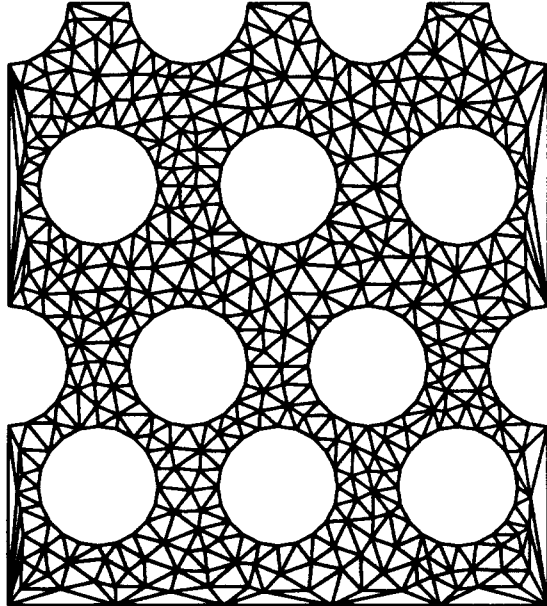


Fig. 17. Triangulation example: 222 edges and a total of 500 points.

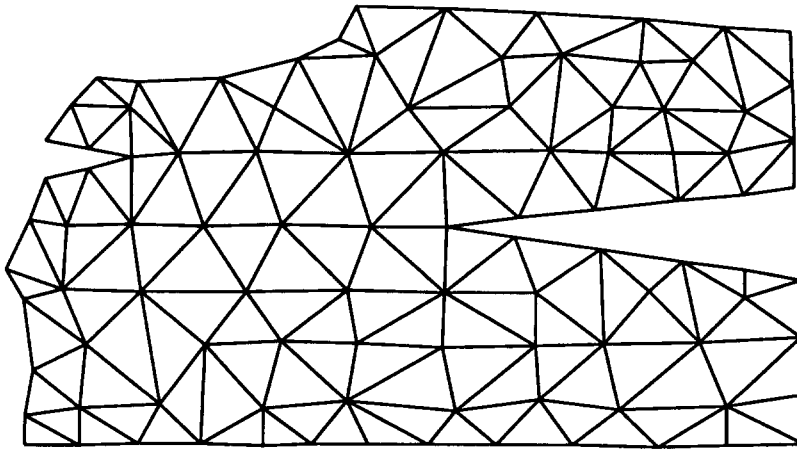


Fig. 18. Triangulation example: 50 edges and a total of 99 points.

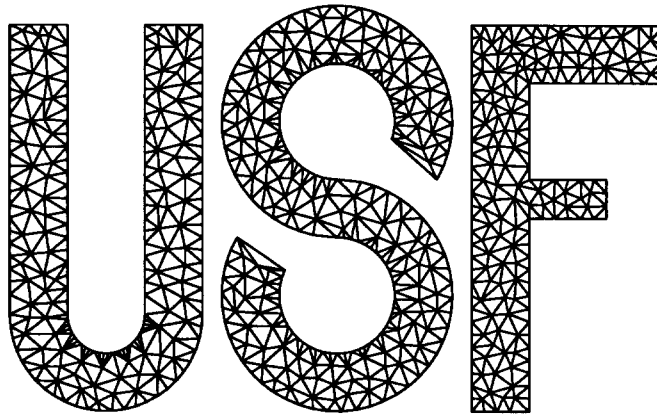


Fig. 19. Triangulation example: 411 edges and a total of 734 points.

lines. Some examples are shown in Figs. 14–19. In Fig. 14 there are 200 points and 200 edges, that is, no interior points, whereas in Fig. 15 the same shape is triangulated with 400 interior points added (a total of 600 points and 200 edges). Figure 16 shows a cheese shaped domain with 219 points and edges. In Fig. 17 the number of points and edges are 500 and 222, respectively. Figure 18 shows the triangulation of a pattern for garment design using 99 points and 50 edges. Finally, in Fig. 19 the University of South Florida logo (USF) is triangulated using 734 points and 411 edges; a place of warning. The presence of co-circular points can, and usually do, create severe numerical problems. In real arithmetic any two theoretically co-circular points are never co-circular practically, due to round off errors. Very fine tuning of the algorithm with clever tolerances is needed to avoid crashing of the program. This problem is not the shortcoming of the algorithm, though. It is the consequence of imprecise computation that is inevitable in floating point arithmetic.

7. CONCLUSIONS

Algorithm and data structure for triangulating multiply connected polygonal domains have been pre-

sented. The method uses a uniform grid data structure to pre-process points and constraint lines. Range searching is applied to each edge to find a point on the right-hand side so that the modified circumcircle criterion is satisfied. A shelling procedure, based on an edge list, puts triangles together so that completeness and correctness are guaranteed. The output data structure is built on the fly without having to store triangles that are outside the valid domain. A point list is used to store data points and the vertices of constraint edges, and is able to incorporate sufficient topological information so that geometric queries, *e.g.*, triangles adjacent to a given triangle, can be answered in a straightforward manner.

Acknowledgements—The work reported in this paper was supported by the Florida High Technology and Industry Council, and by CISI Ingénierie, France.

REFERENCES

1. A. Rockwood, K. Heaton, and T. Davis, Real-time rendering of trimmed surfaces. *Comp. Graph.* **23**, 107–116 (1989).
2. X. Sheng and B. E. Hirsch, Triangulation of trimmed surfaces in parametric space. *Computer-Aided Des.* **24**, 437–444 (1992).

3. H. Okabe, H. Imaoka, T. Tomiha, and H. Niwaya, Three dimensional apparel CAD system. *Comp. Graph.* **26**, 105–110 (1992).
4. L. DeFloriani, B. Falcidieno, and C. Pienovi, Delaunay based representation of surfaces defined over arbitrary shaped domains. *Comp. Vis. Graph. Image Proc.* **32**, 127–140 (1985).
5. J-D. Boissonnat, Shape reconstruction from planar cross sections. *Comp. Vis. Graph. Image Proc.* **44**, 1–29 (1988).
6. M. E. Agishtein and A. A. Migdal, Smooth surface reconstruction from scattered data points. *Comp. Graph.* **15**, 29–39 (1991).
7. M. S. Shephard, Approaches to the automatic finite element generation and control of finite element meshes. *Appl. Mech. Rev.* **41**, 169–185 (1988).
8. K. Ho-Le, Finite element mesh generation methods: a review and classification. *Computer-Aided Des.* **20**, 27–38.
9. W. H. Frey and J. C. Cavendish, Fast planar mesh generation using the Delaunay triangulation. *General Motors Res. Publ.* GMR-4555 (1983).
10. F. Preparata and M. Shamos, *Computational Geometry—An Introduction*, Springer-Verlag, New York (1985).
11. H. Edelsbrunner, *Algorithms in Combinatorial Geometry*, Springer-Verlag, New York (1987).
12. D. T. Lee and A. K. Lin, Generalized Delaunay triangulation for planar graphs. *Discrete and Comp. Geometry*, **1**, 201–217 (1986).
13. S. H. Lo, A new mesh generation scheme for arbitrary planar domains. *Int. J. Num. Meth. Eng.* **21**, 1403–1426 (1985).
14. J. C. Cavendish, Automatic triangulation of arbitrary planar domains for the finite element method. *Int. J. Num. Meth. Eng.* **8**, 679–696 (1974).
15. B. A. Lewis and J. S. Robinson, Triangulation of planar regions with applications. *Comp. J.* **21**, 324–332 (1978).
16. J. L. Nelson, A triangulation algorithm for arbitrary planar domains. *Appl. Math. Mod.* **2**, 151–159 (1978).
17. M. Garey, D. S. Johnson, F. P. Preparata, and R. E. Tarjan, Triangulation of a simple polygon. *Info. Proc. Lett.* **7**, 175–180 (1978).
18. V. Srinivasan and L. R. Nackman, Voronoi diagram for multiply-connected polygonal domains I: Algorithm. *IBM J. Res. Dev.* **31**, 361–372 (1987).
19. V. Srinivasan and L. R. Nackman, Voronoi diagram for multiply-connected polygonal domains II: Implementation and application. *IBM J. Res. Dev.* **31**, 373–381 (1987).
20. N. Sapidis and R. Perucchio, Delaunay triangulation of arbitrary shaped planar domains. *Computer-Aided Geometric Des.* **8**, 421–437 (1991).
21. M. A. Yerry and M. S. Shephard, Finite element mesh generation based on a modified-quadtree approach. *IEEE Comp. Graph. Appl.* **3**, 39–46 (1986).
22. P. L. Bachmann, S. L. Wittchen, M. S. Shephard, K. R. Grice, and M. A. Yerry, Robust, geometrically based, automatic two-dimensional mesh generation. *Int. J. Num. Meth. Eng.* **24**, 1043–1078 (1987).
23. W. J. Schroeder and M. S. Shephard, An $\Theta(n)$ algorithm to automatically generate geometric triangulations satisfying the Delaunay circumsphere criteria. *Eng. Comp.* **5**, 177–193 (1989).
24. A. K. Cline and R. J. Renka, A constrained two-dimensional triangulation and the solution of closest node problems in the presence of barriers. *SIAM J. Num. Anal.* **27**, 1305–1321 (1990).
25. T-P. Fang and L. A. Piegl, Delaunay triangulation using a uniform grid. *IEEE Comp. Graph. Appl.* 36–47 (May, 1993).

APPENDIX

In this section we verify that the range searching technique yields a triangulation that satisfies the modified circumcircle (MCC) criterion. First, it is proven that the first triangle is

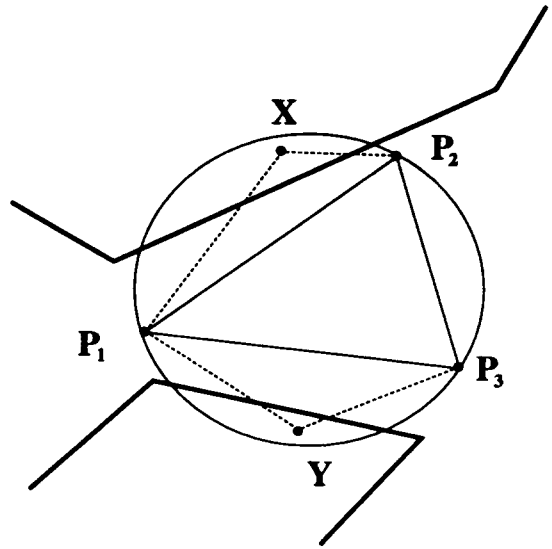


Fig. 20. Proof of the validity of the first triangle $\langle P_1, P_2, P_3 \rangle$.

valid. Consider Fig. 20. The first point selected is P_1 and the first edge is $\langle P_1, P_2 \rangle$. The point satisfying the minimum cosine and the no intersection criteria is P_3 . If there are no constraint edges in the vicinity of the circle (P_1, P_2, P_3) , then no point lies on the right hand side of $\langle P_1, P_2 \rangle$ and inside the circle (P_1, P_2, P_3) because of the minimum cosine criterion. Assume now that the point X lies inside the circle and is to the left of $\langle P_1, P_2 \rangle$. Then since $\overline{P_1 P_2} < \overline{P_1 P_3}$, the complementary arc of (P_1, P_3, P_2) must be less than a semi circular arc. That is, if X lies inside the circle, then $\overline{P_1 X} < \overline{P_1 P_2}$ which contradicts the fact that P_2 is the nearest point to P_1 .

If there are constraint edges in the vicinity of $\langle P_1, P_2 \rangle$, then there are two cases: (1) $\langle P_1, P_2 \rangle$ itself is a constraint edge, and (2) $\langle P_1, P_2 \rangle$ is not a constraint edge. Assume first that $\langle P_1, P_2 \rangle$ is not a constraint edge. Then, if the point X lies inside the circle and to the left of $\langle P_1, P_2 \rangle$, then it must be separated by a constraint edge since it was not returned as

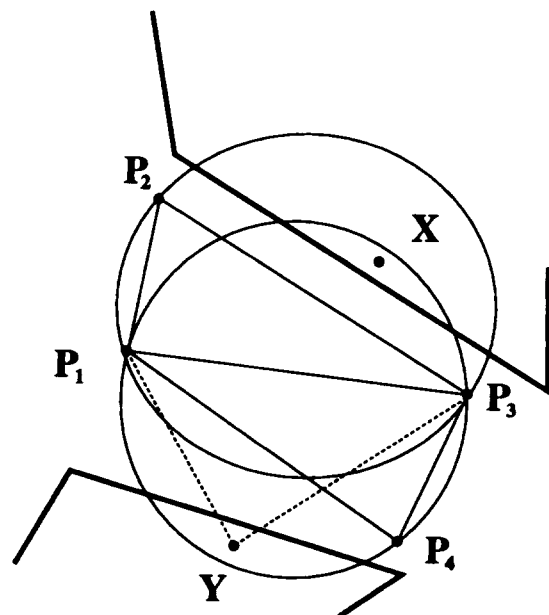


Fig. 21. Proof of the validity of the triangle $\langle P_1, P_3, P_4 \rangle$.

the nearest point (because an intersection was detected). If however, the point Y lies to the right of $\langle P_1, P_2 \rangle$ and inside the circle, then it must also be separated from the triangle by a constraint edge as the lines P_1Y and YP_2 must intersect constraint edges (that is, Y is not returned as the minimum cosine point because intersection was detected).

Assume now that $\langle P_1, P_2 \rangle$ is a constraint edge. Then any point that lies to the left is separated by $\langle P_1, P_2 \rangle$ itself, and the same argument as above applies to points that are to the right. So we have proven that the first triangle obeys the MCC criterion.

Consider the next triangle $\langle P_1, P_3, P_4 \rangle$ shown in Fig. 21. We prove that this triangle satisfies the MCC criterion as well. Assume first that no constraint edges are present in the neighborhood of the two triangles. Then by the range search algorithm's requirement, there is no point to the right of $\langle P_1,$

$P_3 \rangle$ and inside the circle (P_1, P_3, P_4) . Also, there can be no point to the left as the left part of the circular disc (P_1, P_3, P_4) lies entirely inside the circle (P_1, P_2, P_3) that contains no point.

Assume now that constraint edges are present. First of all, if the point X is to the left of $\langle P_1, P_3 \rangle$ and is inside the circumcircle of $\langle P_1, P_3, P_4 \rangle$, then it must be outside the triangle $\langle P_1, P_2, P_3 \rangle$ as illustrated in Fig. 21. Now, since $\langle P_1, P_2, P_3 \rangle$ was a valid triangle, X must be separated from every point of the triangle by a constraint edge. Furthermore, since the lines $\langle P_1, P_4 \rangle$ and $\langle P_3, P_4 \rangle$ do not intersect any constraint edges, the point X has to be separated from the points of the triangle $\langle P_1, P_3, P_4 \rangle$ as well.

Continuing the proof with the edge $\langle P_1, P_4 \rangle$ and relying on the fact that the triangle $\langle P_1, P_3, P_4 \rangle$ is a valid triangle, we can prove validity for $\langle P_1, P_4, P_5 \rangle$, etc.